WEEK 01

Values, types and execution

This week: learn, practise, build

01 · Python

Values, types and execution
Open the weekly notebook beside the slides.

02 · Practice

Predict → run → change → explain
Use small examples from Session 1.

03 · Project

Labelled planning calculation
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Optional lookup · Python Basics 00, 01, 02, 03, 14
Values, types and execution

Python executes statements in order

1

Assign

count = 4
unit_size = 3
Names refer to values.

2

Evaluate

total = unit_size * count
Read values; multiply.

3

Use the result

print(total)
Display the stored value.

Assignment stores a value. Changing an input does not rerun an earlier expression.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
name = value	Assign a value	Keep an input for later
3 * 4	Multiply numbers	Calculate a quantity
print("Total:", total)	Display a value	Label output and units
# explanation	Comment; not executed	Explain an assumption

Python Basics · sections 00, 01, 02, 03, 14 · use only when you need a lookup.

Variables and string literals

```
01 containers = 4
02 print(containers)
03 print("containers")
04 print(type(containers).__name__)
```

1. Predict all three outputs before running
2. Assignment associates a name with a value
3. Change the value and predict again

Quotes create text; an unquoted name retrieves its assigned value.

Numbers and strings behave differently

```
print(3 * 4)
print("3" * 4)
assert 3 * 4 == 12
```

1. How does quoting the first value change the output?
2. Container counts use integer values
3. Keep expectations independent of output

Valid code can still express the wrong operation.

Assign, calculate and print

```
01 description = "Practice meal containers"
02 portions_per_container = 3
03 containers = 4
04 planned = portions_per_container * containers
05 print(description)
06 print("Containers:", containers)
07 print("Planned portions:", planned)
08 assert planned == 12
```

1. Assign input values first.
2. Use an expression to calculate a result.
3. print() displays the stored result.

Keep inputs, calculation and display as separate statements.

Execution order: when is a value recalculated?

```
01 portions_per_container = 3
02 containers = 4
03 planned = portions_per_container * containers
04 containers = 2
05 print("Before:", planned)
06 planned = portions_per_container * containers
07 print("After:", planned)
08 assert planned == 6
```

1. Predict the value printed before recalculation
2. Then predict the value printed after recalculation
3. Rerun the calculation after changing inputs

Changing an input does not rerun an earlier multiplication.

One language, two execution environments

Notebook: .ipynb

Narrative and code cells share a persistent kernel. Run cells and inspect their results.

Source file: .py

A new interpreter reads saved statements from top to bottom. Its output appears in a terminal.

A new script process does not inherit notebook variables.

Save and run a Python file

Context	Command or observation
Working directory	learning_materials/
Terminal	python programs/first_pickup.py
Choose Python	Use your course interpreter; see programs/README.md
Default calculation	3 portions/container × 4 containers
Expected meaning	12 planned portions, not delivered impact

Replace python with your course interpreter; save before rerunning.

Hands-on practice

Week_01.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

Change one value

Use 3 portions/container.
Try 4, then 2 containers.
Predict before running.

Trace execution

Change only the description.
Then change the count.
Which lines need rerunning?

Run a saved file

Copy first_pickup.py.
Set the count to 3.
Save, run, compare.

Work in Week_01.ipynb · Session 2. Predict before running.

Check your work before asking for help

Old output

Rerun assignments and calculation.

Repeated text

Inspect quotes and input types.

Unclear meaning

Name the unit and planning status.

Keep: input → expected result → observed result → explanation.

Project studio

Labelled planning calculation
Short design briefing → most time for your work

The component we add this week

1

Donor declaration

Start from supplied data or the earlier concept.

2

Labelled planning calculation

This week
Implement and explain this part.

3

Review and dispatch rule

Next use
Carry a clear interface and evidence.

Your weekly work builds the reasoning and parts used in the final project.

Trace a declaration into a planning note

1

Declared input

3 planned portions in each specified container

2

Proposed count

4 containers in this practice plan

3

Calculation

3 multiplied by 4 gives 12 planned portions

4

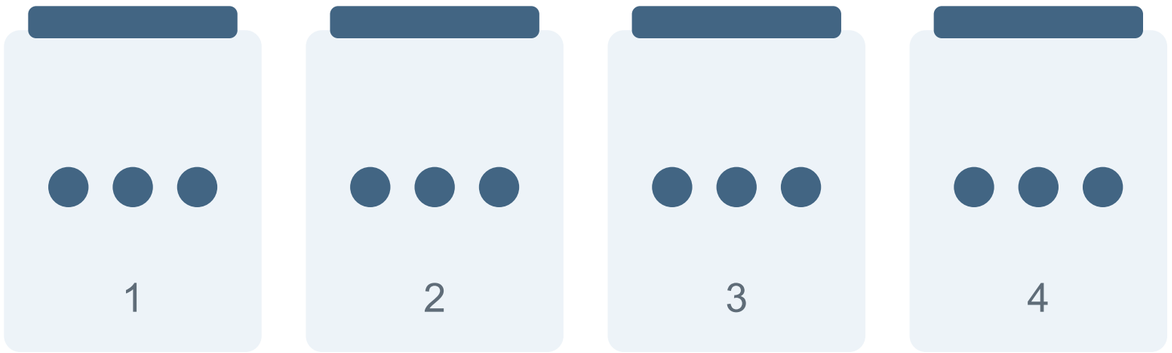
Labelled output

Show the units and retain planning status

Portions per container multiplied by containers gives planned portions.

See the values behind the calculation

4 containers



3 declared portions per container

3 × 4 = 12

Planned portions

A calculation from declared inputs.

Read the diagram, then trace the same steps in code.

Project studio: build, discuss, keep evidence

Build

Bread-pack lab: 2 portions/container. Calculate for 1, 3 and 0 containers.

Discuss

Can zero remain a draft? Which label could mislead a reader?

Keep

Runnable planning note
Predictions and one revised label

Explain the input units, the calculation and one coordinator question.

Before you leave: show one working component

Evidence	Be ready to show
Run	Your actual file or ordered notebook cells
Check	One prediction and its observed result
Explain	One design choice, revision or open question
Connect	Review and dispatch rule

Explain the input units, the calculation and one coordinator question.

Further reading in English

1. ## Python tutorial: numbers and strings

   https://docs.python.org/3/tutorial/introduction.html

2. ## Python tutorial: input and output

   https://docs.python.org/3/tutorial/inputoutput.html

Read one relevant section, then bring an example and a question.